

UD1. EL CLIENTE WEB Y EL ENTORNO DE TRABAJO PROFESIONAL

En esta unidad todavía no se escriben aplicaciones. Se trata de entender dónde se ejecuta lo que se escribe, con qué herramientas se trabaja de verdad en una empresa, y de dejar el entorno montado para el resto del curso.

Contenido de la unidad

1. Qué ocurre cuando se escribe una dirección
 2. El panorama de lenguajes de cliente
 3. HTML y JavaScript: cómo se conectan
 4. DevTools
- Anexo.** Preparación del entorno

1. Qué ocurre cuando se escribe una dirección

Al escribir `www.iesgrancapitan.org` y pulsar Intro, en menos de un segundo pasan siete cosas.

1.1. El viaje de una petición

1. DNS – del nombre a la dirección. El navegador no sabe conectarse a un nombre, solo a una **dirección IP**. El DNS es la guía telefónica que traduce `www.iesgrancapitan.org` a algo como `212.128.1.45`. Antes de preguntar a nadie mira en tres cachés: la suya, la del sistema operativo y la del router. Solo si no está en ninguna sale a preguntar.

2. Conexión – el canal cifrado. Se abre una conexión con ese servidor. Como hoy casi todo es **HTTPS**, encima se negocia una capa de cifrado: el servidor enseña su certificado y el navegador comprueba que lo firma alguien de confianza.

Un error muy extendido sobre HTTPS

Que una web vaya por HTTPS significa dos cosas, y solo dos: que el tráfico va cifrado y que el servidor es de verdad el dueño de ese dominio.

No significa que la web sea legítima. Una web de phishing puede tener HTTPS perfecto: le basta con registrar un dominio parecido al real y pedir su certificado, que es gratis y se consigue en minutos.

Los navegadores ya no marcan las webs seguras, porque es lo normal: avisan de las que no lo son. Por eso lo que hay que mirar no es la barra, sino el nombre del dominio.

3. Petición – el navegador pide. El navegador manda un mensaje de texto. Simplificado:

```
GET /index.html HTTP/1.1
Host: www.iesgrancapitan.org
Accept: text/html
Accept-Language: es-ES
```

Lo importante ahora es el método: **GET** para pedir, **POST** para enviar, **PUT** para actualizar, **DELETE** para borrar.

4. Respuesta – el servidor contesta. Devuelve un **código de estado** y, normalmente, un HTML.

Código	Significa
200	Todo bien, aquí está
301 / 302	Se ha movido, ve a esta otra dirección
404	Eso no existe
403	Existe, pero quien lo pide no tiene permiso
500	La he liado yo, el servidor

La primera cifra ya indica de qué va: 2xx bien, 3xx redirección, 4xx culpa del que pide, 5xx culpa del servidor.

5. DOM – de texto a estructura. Lo que llega es texto plano. El navegador lo lee y construye con él una estructura de objetos en memoria: el **DOM**. A partir de ahí la página deja de ser un archivo y pasa a ser algo vivo que se puede modificar.

6. Recursos – la página no viene sola. Mientras construye el DOM, el navegador encuentra referencias a CSS, imágenes, tipografías y JavaScript, y hace una petición nueva por cada una. Una página normal son decenas o cientos de peticiones.

7. Renderizado – pintar. Con el DOM y los estilos, calcula dónde va cada cosa, la pinta, y ejecuta el JavaScript que la hace interactiva.

1.2. Cliente y servidor: quién hace qué

	Servidor	Cliente (el navegador)
Su código...	se ejecuta y desaparece	se envía entero y se puede leer
¿Se puede leer?	No	Sí, y modificar
Tiene...	los datos reales, las contraseñas, las reglas	lo que el servidor le haya mandado
Sirve para...	la verdad	la experiencia

La regla de oro

No hay que fiarse nunca del cliente. Todo lo que llega al navegador se puede leer, cambiar o saltar, y sin herramientas raras: con las que el navegador ya trae instaladas de fábrica.

Eso no significa que la validación en cliente no sirva. Sirve, y mucho: avisa al usuario al instante sin esperar al servidor. Pero es comodidad, no seguridad.

La seguridad se decide en el servidor, no en el cliente.

Actividad 1 · ¿Dónde se implementa cada cosa, y por qué?

Para cada funcionalidad, decidir si se resuelve en el cliente, en el servidor o en los dos, y por qué. Lo que importa no es la respuesta: es el porqué.

#	Funcionalidad
1	Validar que un email tiene formato correcto
2	Comprobar que ese email no está ya registrado
3	Mostrar u ocultar una sección al pulsar un botón
4	Calcular el total de un carrito de la compra
5	Comprobar que el usuario tiene permiso para ver una página
6	Ordenar una tabla al hacer clic en una columna
7	Aplicar un código de descuento
8	Recordar el modo oscuro que eligió el usuario

Ejemplo: `tienda-tramposa.html` – una tienda con el precio y la validación puestos en el cliente.

Permite pagar 1 € por un producto de 349 € sin instalar nada, y saltarse la validación de un formulario obligatorio.

Lo que este ejemplo demuestra

Ahí no se ha hackeado nada. Se ha editado la copia local de la página, que está en el ordenador de quien la mira y le pertenece.

El fallo no es que la página se pueda editar –eso siempre se puede– sino que el servidor se creyó el precio que le mandó el navegador.

1.3. El navegador por dentro

Un navegador no es un programa: son varios motores trabajando juntos. Dos de ellos importan aquí.

El motor de renderizado parsea el HTML y el CSS, construye el DOM y pinta la página.

Parsear es leer un texto y convertirlo en una estructura con la que se puede trabajar, comprobando de paso que está bien escrito. Lo que llega del servidor es una ristra de caracteres; al parsearla, el navegador construye con ella el DOM. Es lo mismo que hace `json.loads()` en Python con un texto en formato JSON.

Motor	Navegadores
Blink	Chrome, Edge, Opera, Brave, Vivaldi
Gecko	Firefox
WebKit	Safari

Que Blink esté en casi todos explica un problema real del sector: cuando un motor domina, su comportamiento se convierte de hecho en el estándar y las webs se prueban solo ahí. Lo que funciona en Chrome no funciona necesariamente en Safari.

El motor JavaScript ejecuta el código.

Motor	Dónde
V8	Chrome, Edge, Opera y Node.js
SpiderMonkey	Firefox
JavaScriptCore	Safari

V8 es también el motor de Node

Es exactamente el mismo programa ejecutándose fuera del navegador. Por eso JavaScript sirve también para el servidor: alguien cogió el motor y le quitó el navegador de alrededor, con sus restricciones.

JavaScript no es «interpretado» a secas

Se lee por ahí que JavaScript es interpretado y que por eso es lento. Eso era cierto hace quince años.

Hoy el motor empieza interpretando y, en cuanto ve que una función se usa mucho, la compila a código máquina de verdad. Se llama compilación al vuelo, o *JIT*. Por eso Gmail o Figma funcionan dentro de un navegador.

El problema de JavaScript hoy no es la velocidad: es que los errores no aparecen hasta que el programa se está ejecutando. Para eso existe TypeScript.

El sandbox: lo que JavaScript no puede hacer. Literalmente «caja de arena», el cajón donde juegan los niños: juegan lo que quieran, y la arena no sale de ahí.

El código que llega de una web es código de un desconocido ejecutándose en el ordenador del usuario. Que eso sea seguro es el mayor logro de ingeniería del navegador.

No puede	Sí puede
Leer o escribir archivos del disco	Manipular el DOM de su propia página
Ejecutar programas del sistema	Hacer peticiones de red, con restricciones
Acceder a datos de otro dominio libremente	Guardar datos en un almacén propio del dominio
Usar cámara, micrófono o ubicación sin permiso	Pedir permiso al usuario

JavaScript tiene un solo hilo principal en el navegador. Si se bloquea, la página entera se congela – no responde a clics, no hace scroll, nada. De ahí que el lenguaje necesite mecanismos para no bloquear nunca: se pide algo que tarda, se sigue con lo demás, y cuando el resultado está listo el navegador ejecuta la función que se le dejó preparada.

2. El panorama de lenguajes de cliente

2.1. Por qué JavaScript y no otro

De dónde viene lo raro del lenguaje

JavaScript se diseñó en diez días de 1995 y hoy sostiene toda la web. Casi todo lo raro del lenguaje se explica por esas dos frases juntas.

Año	Qué pasó
1995	Netscape quiere un lenguaje dentro de su navegador. Se lo encargan a Brendan Eich con unas dos semanas de plazo. Se llamó Mocha, luego LiveScript, y al final JavaScript por un acuerdo comercial con Sun, la dueña de Java
1996	Microsoft lo copia por ingeniería inversa para Internet Explorer y lo llama JScript. Y añade VBScript, el suyo propio
1997	Con dos versiones incompatibles, se lleva a la organización ECMA para estandarizarlo. Nace ECMAScript
2015	ES6 / ES2015: let , const , funciones flecha, clases, módulos, promesas. Es la frontera de lo que hoy llamamos «JavaScript moderno». Desde entonces sale una versión al año
2022	Se retira Internet Explorer, el 15 de junio. Con él se acaba la guerra de compatibilidad que dominó el desarrollo web durante veinte años

JavaScript y Java no tienen nada que ver. El nombre fue marketing.

Ese plazo de 1995 explica varias rarezas del lenguaje: la conversión automática de tipos, tener `null` y `undefined`, el comportamiento de `this`, el `==` frente al `===`.

Los que perdieron:

Tecnología	Qué era	Cómo murió
VBScript	El lenguaje de scripting de Microsoft	Nunca salió de Internet Explorer
Applets de Java	Programas Java dentro de la página, con plugin	Chrome retiró los plugins en 2015; Java los eliminó en 2018
Flash / ActionScript	El plugin de Adobe para animación e interactividad	Adobe lo retiró el 31 de diciembre de 2020
Silverlight	La respuesta de Microsoft a Flash	Soporte terminado en 2021

Los cuatro tenían algo en común: había que instalar algo, o solo funcionaban en un navegador. JavaScript no.

JavaScript no ganó por ser el mejor. Ganó por ser el único que ya venía dentro. Los demás pedían instalar algo o funcionaban en un solo navegador.

Y una vez dentro, ya no se puede quitar: la web no rompe lo que ya funciona. Por eso el lenguaje solo añade cosas y casi nunca elimina ninguna, y por eso lo que viene después –TypeScript, WebAssembly– no lo sustituye, sino que convive con él.

2.2. Qué es un «lenguaje de guiones»

Script significa *guion*, en el sentido teatral. Y la metáfora es la definición exacta.

Un guion de teatro no se representa solo: necesita actores, escenario y teatro, que ya existen antes que él. Un guion informático es igual: no es un programa autónomo, sino instrucciones para un programa que ya está funcionando. A ese programa se le llama el **anfitrión**.

Guion	Anfitrión que lo ejecuta
JavaScript en una web	El navegador
Un <code>.sh</code> de bash	La shell del sistema
Una macro	Excel
PowerShell	Windows

Comparado con un `.exe`, que arranca solo: un script sin su anfitrión es un archivo de texto que no hace nada.

2.3. Python y JavaScript, cara a cara

	Python	JavaScript
Compilación previa	No	No
Tipado	Dinámico y fuerte	Dinámico y débil
<code>"5" + 5</code>	<code>TypeError</code>	<code>"55"</code>
Ausencia de valor	<code>None</code>	<code>null</code> y <code>undefined</code>
Bloques	Indentación	Llaves
Dónde se ejecuta	Donde se instale	En la máquina del usuario
¿Se ve el código?	No	Sí, entero

La fila del tipado es la importante. Python es dinámico pero **fuerte**: no adivina, protesta. JavaScript es dinámico y **débil**: convierte por su cuenta y sigue adelante.

Actividad 2 · Predecir la salida

¿Qué devuelve cada una de estas cuatro expresiones en la consola del navegador?

```
"5" + 5 · "5" - 5 · [] + {} · 0.1 + 0.2
```

2.4. ECMAScript no es lo mismo que JavaScript

- **ECMAScript** es la especificación: el documento que define la sintaxis y el comportamiento.
- **JavaScript** es la implementación en un navegador, más un montón de cosas que no son ECMAScript.

Y ese montón sorprende: el DOM, `fetch`, `localStorage`, los eventos, `setTimeout`... nada de eso está en ECMAScript. Son APIs del navegador.

ECMAScript da el lenguaje; el navegador da el mundo sobre el que actuar. Por eso el mismo JavaScript en Node no tiene `document` ni `alert`.

Quién decide qué entra. Un comité llamado **TC39**, dentro de ECMA, con representantes de Google, Apple, Mozilla, Microsoft y otros. Cada propuesta recorre cinco fases, desde «una idea suelta» hasta «implementado en dos motores y con pruebas», y solo al llegar al final entra en la siguiente versión.

Sale una versión al año, cada junio, y casi nunca se elimina nada: por eso `var` sigue funcionando aunque ya no deba usarse. El lenguaje solo añade, prácticamente nunca quita.

2.5. ¿Puedo usar esto? La etiqueta Baseline

Durante veinte años, escribir JavaScript era negociar con navegadores incompatibles. Con la muerte de Internet Explorer eso se acabó, y hoy la pregunta se responde mirando la tabla de compatibilidad de MDN.

Baseline es una etiqueta que aparece en MDN. Mira cuatro navegadores –Chrome, Edge, Firefox y Safari– y da uno de tres resultados:

Etiqueta	Significa	Qué implica
Limited availability	Falta en alguno de los cuatro	No usarla en producción
Newly available	Ya funciona en los cuatro, desde una fecha concreta	Se puede usar, sabiendo que quien no actualice se queda fuera
Widely available	Han pasado 30 meses desde entonces	Se usa sin pensar

Esos 30 meses son el margen para que la gente actualice. Es decir: una característica pasa sola de «newly» a «widely» con el tiempo.

Cómo se mira: se entra en <https://developer.mozilla.org>, se busca la característica –por ejemplo `Array.at`– y el recuadro aparece justo debajo del título, sin tener que bajar. Al hacer clic se despliega, con los cuatro navegadores y una marca de sí o de no en cada uno. Al final de la página, Browser compatibility da la versión exacta de cada navegador.

MDN, mejor en inglés

La traducción al español la hace la comunidad, es parcial y va meses o años por detrás. Las páginas más recientes no están traducidas, así que el buscador en español no las encuentra.

En cualquier página traducida aparece arriba el botón **«Always switch to English»**. Un clic y queda resuelto.

La versión en español oculta justo lo reciente, que es lo que hay que comprobar.

Otra herramienta: <https://webstatus.dev>, el panel oficial de Baseline, con buscador y filtros.

Actividad 3 · ¿Puedo usar esto?

De estas seis características: qué etiqueta Baseline tiene cada una en MDN –limited, newly o widely– y desde qué mes y año.

`Array.prototype.at()` · `structuredClone()` · `Array.prototype.toSorted()` · `Object.groupBy()` · `Promise.withResolvers()` · `Temporal`

Y después, dos preguntas:

- A.** ¿Hay algún patrón en el orden de esa lista?
- B.** Ninguna etiqueta es para siempre. ¿Cuál va a cambiar antes, y por qué?

2.6. JavaScript, TypeScript y WebAssembly

Los tres se ejecutan en el cliente, pero no son tres alternativas al mismo problema.

JavaScript es el único que el navegador ejecuta de forma nativa. Todo lo demás acaba convertido en JavaScript o convive con él.

TypeScript (Microsoft, 2012) no es un lenguaje distinto: es JavaScript más un sistema de tipos.

TypeScript no existe en el navegador

Los tipos se comprueban al compilar y luego se borran. Lo que llega al navegador es JavaScript normal, sin una sola anotación de tipo.

Consecuencia: un tipo de TypeScript no valida nada mientras se ejecuta. Protege del error tonto al escribir el código, no del dato inesperado que devuelve una API.

WebAssembly (Wasm), estándar desde 2017, es un formato binario que el navegador ejecuta a velocidad casi nativa. No se escribe a mano: se compila desde C, C++, Rust o Go. Tres cosas que casi todo el mundo dice mal:

1. No sustituye a JavaScript. Convive con él
2. No accede al DOM directamente. Para tocar la página tiene que llamar a JavaScript
3. Solo compensa en cálculo intenso. Para una web normal no aporta nada

Dónde se usa de verdad: Figma, AutoCAD Web, Photoshop Web, edición de vídeo en el navegador, emuladores.

3. HTML y JavaScript: cómo se conectan

Esta parte trata de cómo se conecta un archivo JavaScript con una página HTML: dónde se coloca la etiqueta `<script>`, qué atributos lleva, y por qué esa decisión, que parece menor, determina si el código encuentra la página o no.

3.1. Las dos formas de incluir JavaScript

En línea, dentro de la propia etiqueta:

```
<script>
  console.log("hola");
</script>
```

Externo, en un archivo aparte:

```
<script src="titulo.js"></script>
```

En la práctica profesional, siempre externo. Por cuatro motivos, en orden de importancia:

1. Se cachea. El navegador se guarda **app.js** y no vuelve a descargarlo en cada página. Un `<script>` en línea viaja dentro del HTML cada vez.
2. Se reutiliza en varias páginas.
3. Separa las capas, que es el último apartado de esta unidad.
4. El editor da autocompletado y avisos de error en un **.js**. Dentro de un `<script>` de HTML, regular.

Un `<script>` con `src` ignora lo de dentro

Si un `<script>` tiene `src`, lo que se escriba dentro se ignora. En silencio, sin ningún aviso.

```
<script src="app.js">
  console.log("esto no se ejecuta nunca");
</script>
```

O carga el archivo, o ejecuta lo de dentro. Las dos cosas a la vez, no.

3.2. El problema del orden

Es la idea central de esta parte, y la que explica los errores que vienen después.

El navegador lee el HTML de arriba abajo, construyendo el DOM según avanza. Cuando se encuentra un `<script>` clásico:

1. Para de construir el DOM
2. Descarga el archivo, si es externo
3. Lo ejecuta entero
4. Y solo entonces sigue leyendo el HTML

A eso se le llama **script bloqueante**, y es el comportamiento por defecto desde 1995. Nunca se ha cambiado, por compatibilidad.

ANTES DE SEGUIR: EL ENTORNO

A partir de aquí los ejemplos se abren en el ordenador, no solo se leen. Hace falta tener descomprimida la carpeta de ejemplos y VS Code con sus dos extensiones.

Está todo en el anexo del final del documento, «Preparación del entorno». Los ejemplos 01, 02 y 03 se abren con doble clic; del 04 en adelante hace falta Live Server.

Ejemplo: `01-script-roto.html` – el script en el `<head>`, sin atributos, y el `<h1>` más abajo, en el `<body>`.

Para ver la consola: **clic derecho en la página → Inspeccionar → pestaña Console**. (Atajo: *F12* en Windows y Linux, *Cmd + Option + J* en Mac.)

`null` casi nunca significa «lo has escrito mal». Significa «aún no está ahí».

3.3. Las dos soluciones

La histórica: el `<script>` al final del `<body>`.

Ejemplo: `02-script-final-body.html` – exactamente el mismo `titulo.js`, con la etiqueta movida justo antes de `</body>`.

Funciona, porque para cuando se ejecuta el DOM ya está construido. Es el arreglo que aparece en la mayoría de tutoriales y sigue siendo válido. Pero desperdicia tiempo: el navegador no empieza a pedir el archivo hasta que llega abajo del todo, así que durante toda la lectura del HTML la red ha estado parada.

La moderna: `defer`.

Ejemplo: `03-script-defer.html` – otra vez el mismo `titulo.js`, con la etiqueta de vuelta en el `<head>` y con `defer`.

Funciona, y además descarga en paralelo.

La analogía

Es como pedir una pizza mientras se limpia la casa.

Con el script al final: se limpia toda la casa y, al terminar, se llama a la pizzería. Y ahí quedan media hora de brazos cruzados.

Con **defer**: se llama nada más empezar, y se limpia mientras la traen.

En los dos casos se come. Pero en uno se ha esperado de más para nada.

02 · el `<script>` al final del `<body>`

lee el HTML

descarga titulo.js

lo ejecuta

Una cosa detrás de otra: hasta que no termina de leer, no pide el archivo.

03 · `<script defer>` en el `<head>`

ve el `<script>`

descarga titulo.js

sigue leyendo el HTML

lo ejecuta

Las dos cosas a la vez: cuando termina de leer, el archivo ya está.

aquí acababa el 02

tiempo

En las DevTools, pestaña Network, con *Disable cache* marcado y recargando la página: en el 03 el HTML y `titulo.js` se descargan a la vez; en el 02 la barra de `titulo.js` empieza mucho más tarde.

3.4. defer, async y type="module"

Atributo	¿Cuándo descarga?	¿Cuándo ejecuta?	¿Orden entre scripts?	¿Bloquea el parseo?
(ninguno)	Al encontrarlo	Inmediatamente	Sí, el del documento	Sí
defer	En paralelo	Cuando el HTML está parseado	Sí, el del documento	No
async	En paralelo	En cuanto termina de descargarse	No. Impredecible	Al ejecutar
type="module"	En paralelo	Como defer	Sí	No

Sobre **async**: con dos scripts marcados como **async** de los que uno necesita al otro, funcionará unas veces sí y otras no, según lo rápido que descargue cada uno. Es difícil de diagnosticar, porque no falla siempre.

La regla de este curso

type="module". Siempre.

Para una librería antigua de un CDN que no sea módulo, **defer**. Para algo que no dependa ni del DOM ni de otro script –analítica, un chat–, **async**. Y **<script>** a secas, nunca.

Un módulo ya incluye **defer**, y además trae dos cosas más. Poner **defer** cuando se puede poner **module** es quedarse con la mitad.

Un detalle fácil de pasar por alto: **defer** en un **<script>** en línea, sin **src**, se ignora. Solo tiene efecto sobre scripts externos.

3.5. Qué hace **type="module"**

Cuatro cosas:

1. Se comporta como **defer** sin pedirlo
2. Activa el modo estricto, sin poder desactivarlo
3. Tiene ámbito propio: sus variables no contaminan el global. Dos módulos pueden tener cada uno su **const nombre** sin pisarse
4. Habilita **import** y **export**

Antes había que pedir el modo estricto a mano

Fuera de un módulo se activa escribiendo **"use strict"**; como primera línea del archivo, o dentro de una función para aplicarlo solo a ella.

Es una cadena de texto suelta porque los navegadores antiguos de 2009 la leían como un texto sin uso y seguían, en vez de dar error de sintaxis. Si se pone después de cualquier código, se ignora.

En un módulo no hace falta escribirla, así que se ve en código antiguo pero ya no se escribe.

Ejemplo: **04-modulos/index.html** – se abre primero con doble clic, para ver el error, y después con Live Server.

El modo estricto. JavaScript de 1995 era extremadamente permisivo: en vez de dar error, muchas cosas fallaban en silencio y el programa seguía adelante. En 2009 no se podía arreglar sin romper

media web, así que se creó un modo aparte al que uno se apunta voluntariamente. Y dentro de un módulo está activado siempre.

El modo estricto no tiene nada que ver con los tipos

Viniendo de Python esto confunde: no es tipado estricto. Es «sé estricto conmigo cuando meta la pata»: convierte fallos silenciosos en errores visibles.

El cambio principal es qué pasa con una variable sin declarar:

```
function contar() {  
  contdor = 5;      // errata: querías escribir "contador"  
}
```

Ejemplo: `05-modo-estricto/` – dos páginas que cargan exactamente el mismo `contador.js`, con una errata a propósito. Una lo carga con `defer` y la otra con `type="module"`.

Se abren en dos pestañas y se comparan las consolas.

3.6. DOMContentLoaded y load

Dos eventos que se confunden a menudo. Aparecen en código antiguo envolviéndolo todo:

Evento	Se dispara cuando...
DOMContentLoaded	El HTML está parseado y el DOM completo. No espera a las imágenes
load	Ha terminado de cargar todo: imágenes, iframes, tipografías

DOMContentLoaded no hace falta en este curso, porque un módulo ya espera al DOM: `type="module"` incluye `defer`, y `defer` garantiza que el DOM está listo.

`load` es otra cosa, y sí sigue haciendo falta a veces, se usen módulos o no. Un módulo garantiza que el `<img>` existe, no que la imagen se haya descargado: para saber su tamaño real, `naturalWidth` vale `0` hasta que llega.

3.7. Separación de capas

Tres capas, tres responsabilidades:

Capa	Lenguaje	Responde a
Contenido	HTML	¿Qué hay?
Presentación	CSS	¿Qué aspecto tiene?

Capa	Lenguaje	Responde a
Comportamiento	JavaScript	¿Qué hace cuando el usuario interactúa?

Qué las rompe.

1. Manejadores de eventos escritos en el HTML.

```
<!-- mal -->
<button onclick="document.querySelector('#caja').style.display='none'">Cerrar</button>

<!-- bien -->
<button id="cerrar">Cerrar</button>
```

```
document.querySelector("#cerrar").addEventListener("click", () => {
  document.querySelector("#caja").classList.add("oculto");
});
```

2. Estilos escritos desde JavaScript.

```
caja.style.display = "none";    // mal: el aspecto vive en el JS
caja.classList.add("oculto");   // bien: el JS decide el estado, el CSS decide cómo se ve
```

3. Texto visible incrustado en el JavaScript.

Los mensajes que lee el usuario escritos dentro del código, en vez de estar en el HTML.

```
// mal: el texto vive en el JavaScript
aviso.textContent = "No hay resultados para esa búsqueda";

// bien: el texto está en el HTML y el JavaScript solo lo enseña
// <p id="aviso" hidden>No hay resultados para esa búsqueda</p>
aviso.hidden = false;
```

El contenido es del HTML; el JavaScript decide cuándo se enseña, no qué dice. Si los textos están repartidos por el código, traducir la aplicación obliga a buscarlos uno a uno dentro del JavaScript.

Ejemplos: `06-capas-mal.html` y `07-capas-bien/` – hacen exactamente lo mismo, uno con las capas mezcladas y otro con las tres separadas.

El primero funciona perfectamente. No tiene ningún error. Y aun así está mal. La diferencia se ve respondiendo a la misma pregunta con cada uno delante: *si hubiera que hacer que además se desvaneciera en medio segundo, y que el mismo botón funcionara en otras tres páginas, ¿qué archivos habría que tocar?*

Cada capa se cambia sin tocar las otras. Rediseñar es tocar CSS. Cambiar un texto es tocar HTML. Arreglar un comportamiento es tocar JS.

Si están mezclados, cualquier cambio los toca los tres, y cada cambio puede romper algo que no se estaba mirando.

3.8. Dudas frecuentes

¿Por qué tanta gente pone el script al final del `<body>`? Porque durante quince años fue la única solución. `defer` existe desde hace mucho, pero las costumbres duran más que los navegadores. Las dos funcionan; `defer` es mejor porque descarga en paralelo.

¿Se pueden poner varios `<script>`? Todos los que hagan falta. Con `defer` se ejecutan en el orden en que aparecen en el documento. Con `async`, en el orden en que vayan llegando, que no se puede predecir.

¿Y por qué no meter todo el JavaScript dentro del HTML y ya está? Porque no se cachea, no se reutiliza, el editor no ayuda y el archivo crece hasta ser inmanejable. Funciona para un ejemplo de diez líneas y para nada más.

¿El CSS también bloquea? Sí. El CSS bloquea el renderizado, aunque no el parseo: el navegador sigue leyendo el HTML, pero no pinta hasta tener los estilos.

Si un módulo ya espera al DOM, ¿load no sirve para nada? Sirve, y para otra cosa. El módulo garantiza que el DOM está construido: los elementos existen. `load` garantiza que los subrecursos han terminado de descargarse: imágenes, iframes, tipografías. Un módulo da el `<img>`; no da el píxel.

4. DevTools

Todo lo anterior se ha explicado mirando código. A partir de aquí se puede mirar el programa mientras se ejecuta.

Las DevTools son las herramientas que el navegador trae de fábrica para eso. Se abren con **clic derecho en la página → Inspeccionar**. (Atajo: *F12* en Windows y Linux, *Cmd + Option + I* en Mac.)

4.1. Qué panel sirve para qué

Hay doce paneles y en este módulo se usan cuatro.

Panel	Para qué sirve
Elements	Ver y modificar el DOM y los estilos ya aplicados
Console	Ejecutar JavaScript sobre la página cargada, y ver lo que el código escribe
Sources	Ver el código y parar la ejecución para mirar dentro
Network	Ver las peticiones reales, con sus cabeceras y sus tiempos
Application	El almacenamiento del navegador y las cookies

Lo primero, antes de nada

Subir el tamaño de letra del panel: clic dentro de las DevTools y **Cmd** o **Ctrl** más **+**. La tipografía de las DevTools es independiente de la de la página, y por defecto es diminuta.

4.2. La consola es más que `console.log`

Instrucción	Qué hace
<code>console.table(datos)</code>	Pinta una lista de objetos como tabla, con sus columnas
<code>console.dir(elemento)</code>	Muestra un elemento del DOM como objeto, con todas sus propiedades
<code>console.group()</code> · <code>console.groupEnd()</code>	Agrupar y pliega mensajes, útil dentro de un bucle
<code>console.time()</code> · <code>console.timeEnd()</code>	Mide cuánto tarda un trozo de código
<code>console.count()</code>	Cuenta las veces que se pasa por ahí
<code>console.error()</code> · <code>console.warn()</code>	Salen en rojo o en amarillo, y con la traza de llamadas

Ejemplo: `08-consola/`

4.3. Puntos de interrupción: parar el programa y mirar dentro

Un **punto de interrupción** congela el programa en una línea concreta y permite ver todas las variables en ese instante, con su valor y su tipo, sin haber escrito ni un `console.log`.

Se pone en el panel Sources: se abre el archivo y se hace clic en el número de línea. Al recargar, la ejecución se detiene ahí y la página se queda a medias.

Entonces aparecen tres zonas que hay que aprender a leer:

Zona	Qué muestra
Scope	Las variables vivas en ese punto, con su valor y su tipo
Call Stack	Quién ha llamado a quién para llegar hasta ahí
Watch	Expresiones que se quieren vigilar en cada parada

Y cuatro botones para avanzar: continuar hasta el siguiente punto, saltar por encima de la línea, entrar dentro de la función que se llama, y salir de la función actual.

Ejemplo: `09-depurar-total/`

Un carrito de 10, 15 y 10 euros cuyo total debería ser 35, y no lo es. La consola no muestra ningún error: el código hace exactamente lo que se le ha pedido.

Con un punto de interrupción en la línea que acumula el total se puede mirar el panel Scope en cada vuelta del bucle.

También existe la instrucción **debugger**;: escrita dentro del código, detiene la ejecución en ese punto siempre que las DevTools estén abiertas. Es útil para código que se genera dinámicamente, pero hay que acordarse de quitarla.

Dos variantes menos conocidas

Punto de interrupción condicional: clic derecho sobre el número de línea. Se escribe una condición y solo para cuando se cumple. Sirve para llegar al elemento 300 de un bucle sin pasar 299 veces.

Logpoint: también con clic derecho. Escribe un mensaje en la consola sin parar y sin tocar el código. Es un `console.log` que no ensucia el archivo y que no hay que acordarse de borrar.

4.4. Network y Application

Network ya apareció en el primer apartado, cuando se vio que una página son decenas de peticiones. Dos cosas concretas que se usan a diario:

- **Disable cache**, marcado mientras se desarrolla. Si no, se puede pasar media hora depurando un archivo que el navegador no ha vuelto a pedir.
- Cada petición tiene cabeceras, código de estado y cuerpo, y ahí se ven de verdad.

Application muestra lo que el navegador guarda: `localStorage`, `sessionStorage` y las cookies. Se puede ver y borrar desde ahí.

4.5. Dudas frecuentes

¿Esto no es más lento que poner `console.log`? Al principio sí. A la tercera vez, no. Un `console.log` responde a una pregunta que ya sabías hacer; un punto de interrupción enseña todo el estado, incluidas las variables que no se te había ocurrido mirar. Y no hay que acordarse de borrarlo.

¿Se puede modificar el código desde Sources? Sí, y los cambios se ejecutan, pero no se guardan: al recargar vuelve el archivo original. Sirve para probar una idea en caliente, no para trabajar.

¿Por qué mi punto de interrupción no para? Casi siempre porque el código ya se había ejecutado antes de ponerlo: hay que recargar. Si aun así no para, suele ser que el archivo abierto no es el que la página está usando de verdad.

¿Esto vale para el código del servidor? No. Las DevTools ven lo que llega al navegador. El código de servidor se depura con las herramientas del servidor, que se ven en otro módulo del ciclo.

¿Los nombres de los paneles son siempre los mismos? No: cambian entre versiones del navegador, y entre navegadores. Los conceptos –parar, mirar el ámbito, avanzar paso a paso– son los mismos en todos; los menús, no.

Actividad 4 • Encontrar el fallo sin leer el código

10–practica–depurar/ calcula la media de tres notas –6, 8 y 7– y debería dar 7. Da otra cosa, y tampoco hay ningún error en rojo.

La pregunta no es cuál es el fallo, sino en qué vuelta exacta se estropea el resultado y por qué.

Para saber más

- [Overview of HTTP – MDN](#)
- [HTTP response status codes – MDN](#)
- [Populating the page: how browsers work – MDN](#)
- [JavaScript – MDN](#)
- [Baseline y compatibilidad – MDN](#)
- [Web Platform Status](#) – el panel oficial de Baseline
- [<script>: the Script element – MDN](#)
- [JavaScript modules – MDN](#)
- [Strict mode – MDN](#)

- [EventTarget.addEventListener\(\) – MDN](#)
- [Documentación de VS Code](#)

Anexo · Preparación del entorno

Lo mínimo para abrir los ejemplos y escribir código: un editor con dos extensiones, una forma fiable de abrir las páginas y una manera ordenada de guardar el trabajo.

A.1. El editor y sus extensiones

VS Code es un editor de texto con dos cosas que lo separan del Bloc de notas: entiende el lenguaje que se está escribiendo y admite extensiones.

Una extensión es un programa escrito por otra persona que se instala dentro del editor y añade una función: colorear un lenguaje, formatear al guardar, levantar un servidor. Se instalan desde el panel de extensiones, el icono de los cuadrados en la barra lateral (*atajo: Ctrl + Mayús + X en Windows y Linux, Cmd + Mayús + X en Mac*), buscando por nombre y pulsando Install.

En este módulo se usan dos:

Extensión	Para qué
Live Server	Abrir las páginas con un servidor local en vez de con doble clic
Prettier – Code formatter	Dar formato al código automáticamente al guardar

Y tres opcionales, por comodidad:

Extensión	Para qué
Error Lens	Muestra el error en la misma línea en la que está, sin pasar el ratón por encima
Path Intellisense	Autocompleta las rutas al escribir un <code>src</code> o un <code>href</code>
vscode-icons	Iconos por tipo de archivo en el explorador

Una extensión también es código de un desconocido

Una extensión se ejecuta dentro del editor, con acceso a los archivos que hay abiertos. Antes de instalar una conviene mirar quién la publica y cuántas instalaciones tiene: las dos de este módulo tienen millones y editor oficial identificado.

La misma precaución vale para cualquier cosa que se instale en un proyecto.

A.2. Por qué hace falta un servidor local

En el apartado 3 aparece un problema sin resolver: un archivo con `type="module"` abierto con doble clic no funciona. En la consola sale un error de CORS y el script no llega a ejecutarse.

El motivo es el **origen**. Al abrir un archivo con doble clic, la dirección empieza por `file://` y el navegador considera que esa página no tiene origen. Los módulos exigen un origen real, así que el navegador se niega antes de empezar. Al servirla por HTTP, la dirección pasa a ser `http://localhost:5500/...`, hay origen, y todo funciona.

Localhost es el nombre reservado para el propio ordenador. Nada sale a internet: el navegador pide la página a un programa que está funcionando en la misma máquina.

Con Live Server instalado, se hace **clic derecho sobre el archivo .html → Open with Live Server**. Se abre el navegador con la página servida, y cada vez que se guarda un cambio, la página se recarga sola.

Abrir la carpeta, no el archivo

Live Server sirve la carpeta que esté abierta en el editor. Por eso conviene abrir en VS Code la carpeta de la unidad –*Archivo → Abrir carpeta*– y no el archivo suelto: así las rutas relativas a imágenes, estilos y módulos funcionan igual que funcionarán en un servidor de verdad.

A.3. Formateo automático del código

Formatear es decidir cómo se ve el código: comillas simples o dobles, dos espacios o cuatro, punto y coma al final o no. No cambia lo que el programa hace.

Cada persona tiene sus preferencias, y eso da problemas al trabajar en equipo: al revisar un cambio aparecen modificadas muchas líneas en las que en realidad no ha cambiado nada, solo el sangrado. Por eso el formato lo aplica una herramienta. **Prettier** usa las mismas reglas para todos los archivos y para todo el equipo.

Con la extensión instalada, hacen falta tres ajustes. Se abren desde **Archivo → Preferencias → Configuración** en Windows y Linux, o **Code → Ajustes → Ajustes** en Mac (*atajo: Ctrl + ,* en Windows y Linux, *Cmd + ,* en Mac), y cada uno se localiza escribiendo su nombre en el buscador de arriba:

Ajuste	Valor	Para qué
Default Formatter	Prettier - Code formatter	Decide qué herramienta da formato. Sin elegirlo, VS Code tiene dos disponibles para HTML y, al guardar, no usa ninguna
Format On Save	Marcado	Da formato cada vez que se guarda
Auto Save	off u onFocusChange	Con <code>afterDelay</code> el archivo se guarda solo cada poco tiempo, y en ese caso no se da formato. <code>onFocusChange</code> guarda al pasar a otra ventana, por ejemplo al navegador

Para comprobarlo: se desordena a propósito el sangrado de un archivo y se guarda (*Ctrl + S* en Windows y Linux, *Cmd + S* en Mac), o se cambia al navegador si el guardado es `onFocusChange`. El sangrado vuelve a su sitio.

Si no ocurre nada, se prueba a dar formato a mano: **clic derecho dentro del archivo → Dar formato al documento**. Si aparece un aviso preguntando qué formateador usar, se elige Prettier; eso significa que faltaba el primer ajuste. Si a mano funciona pero al guardar no, el problema está en uno de los otros dos.

Un formateador no es un corrector

Prettier no busca errores ni opina sobre lo que hace el código: reescribe su aspecto y nada más. Un programa con una variable mal escrita seguirá estando mal después de formatearlo, solo que bien alineado.

A.4. Cómo se organiza el trabajo del módulo

Son detalles, y evitan tropiezos constantes:

- Una carpeta por unidad, dentro de una carpeta del módulo, en un sitio fijo del disco.
- Descomprimir el zip antes de trabajar. Si se abre el archivo desde dentro del zip, los cambios se pierden o se guardan en una carpeta temporal.
- Fuera de la carpeta de descargas. Es la que se vacía sin mirar cuando falta espacio.
- Nombres de archivo y de carpeta sin espacios, sin acentos y en minúsculas. En una URL, un espacio se convierte en `%20` y una ñe en algo peor. La convención del curso es la de los ejemplos: `06-capas-mal.html`.

Mayúsculas: funciona en tu equipo y falla en el servidor

Windows y macOS tratan igual `Logo.PNG` y `logo.png`. Linux, que es donde acaba alojada la mayoría de las webs, no: ahí son dos archivos distintos y la imagen no aparece.

Desconcierta porque en local todo iba bien. Con nombres en minúsculas no ocurre nunca.

Actividad 5 • Dejar el entorno montado

El entorno queda montado cuando `04-modulos/index.html` funciona servido con Live Server y el editor da formato al guardar.

Con eso delante: qué cambia entre abrir ese archivo con doble clic y abrirlo con Live Server, y por qué; qué ha hecho el editor al guardar; y en qué dirección sirve Live Server la página.

A.5. Dudas frecuentes

Live Server no aparece al hacer clic derecho. La extensión no está instalada o el editor no se ha reiniciado después de instalarla.



La página no refleja el último cambio. Casi siempre es que el archivo no está guardado. Si está guardado y sigue igual, es la caché del navegador: recarga forzada con **Ctrl + Mayús + R** en Windows y Linux, **Cmd + Mayús + R** en Mac.

¿Se puede usar otro editor? Sí, pero el material y las indicaciones de clase están hechos con VS Code, y es el editor más extendido en el sector.

RA1. Selecciona las arquitecturas y tecnologías de programación sobre clientes web, identificando y analizando las capacidades y características de cada una.

CE.a. Se han caracterizado y diferenciado los modelos de ejecución de código en el servidor y en el cliente web.

CE.b. Se han identificado las capacidades y mecanismos de ejecución de código de los navegadores web.

CE.c. Se han identificado y clasificado los lenguajes de programación en clientes web.

CE.d. Se han reconocido las particularidades de la programación de guiones y las diferencias con la programación tradicional.

CE.e. Se han verificado los mecanismos de integración de los lenguajes de marcas con los lenguajes de programación de clientes web.

CE.f. Se han utilizado herramientas y entornos para facilitar la programación, prueba y depuración de código en clientes web.